

TRACKER · INTERNAL GUIDE

# Offer Settings Explained & End-to-End Testing Guide

What every setting on the Edit Offer screen does, what actually happens when you turn it on or off, and how to test Advertiser → Offer → Partner → Click → Conversion → Approval → Payout → Partner Postback.

Prepared 8 October 2026 from the Tracker source code (frontend + api-backend), not from the on-screen labels alone.

Every setting carries a status badge: **ENFORCED** the tracker acts on it · **PARTIAL** acts on it with limits ·

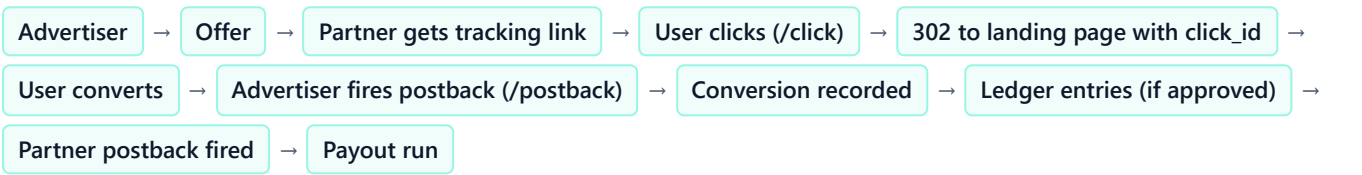
**SAVED ONLY** stored on the offer but nothing happens yet.

# Contents

|                                                                                   |                                                  |
|-----------------------------------------------------------------------------------|--------------------------------------------------|
| 1. The big picture: how a conversion flows                                        | 5. Attribution — every setting                   |
| 2. Tracking & Controls — Linking Type, Deep Links                                 | 6. Creatives                                     |
| 3. Postback Configuration — Security Code, Publisher Postback                     | 7. Email                                         |
| 4. Revenue & Payout (Events) — Fire Partner Postback, Manual Approval, Duplicates | 8. Step-by-step testing guide                    |
|                                                                                   | 9. Settings test matrix                          |
|                                                                                   | 10. Final verification checklist & known gotchas |

## 1. The big picture: how a conversion flows

Every setting in this guide changes one step of this chain. Knowing the chain makes each setting easy to reason about.



| Step             | What the system does                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Click            | The partner's link ( /click?offer_id=...&pub_id=... ) hits the <b>tracking server (port 4002)</b> . It checks the offer is active, the partner is allowed, geo/device targeting, traffic controls, caps and uniqueness, then creates a <b>click_id</b> , saves the click (in the background via a worker) and redirects the user to the offer's landing page, replacing macros like {click_id} in that URL. |
| Conversion       | The advertiser's server calls /postback?click_id=...&txn_id=...&status=approved (or loads the /pixel / /iframe ). Tracker finds the click, checks the security code, duplicates, attribution window, click-to-conversion time, manual approval, caps and throttle, then stores the conversion as <b>approved</b> , <b>pending</b> or <b>rejected</b> .                                                      |
| Money            | Only an <b>approved</b> conversion writes to the append-only ledger: a <b>publisher earning</b> (partner payout) and an <b>advertiser billing</b> (revenue). Pending conversions write nothing until approved.                                                                                                                                                                                              |
| Partner postback | After an approved conversion, a background worker calls the partner's configured postback URL so the partner knows they earned.                                                                                                                                                                                                                                                                             |
| Payout           | A payout run converts each partner's approved balance into a payout record and a ledger debit.                                                                                                                                                                                                                                                                                                              |

## 2. Tracking & Controls (Image 1)

### Linking Type — Redirect Linking vs Redirect + Direct Linking

SAVED ONLY

What it means in the industry:

- **Redirect Linking** — the partner sends traffic to the *tracking link*. The tracker records the click on its own server, then redirects (HTTP 302) the user to the advertiser's landing page. This is the classic, most reliable method because the click is recorded server-side before the user ever reaches the advertiser.
- **Direct Linking** — the partner links *straight to the advertiser's website* (no tracking redirect). Platforms such as Everflow make this work by placing a JavaScript snippet on the advertiser's landing page that records the click from the browser. It is used where redirects are not allowed (e.g. Google Ads / Meta ads policies that require the final URL domain to match) or to avoid redirect latency.
- **Redirect + Direct Linking** — the offer allows both: partners may use the tracking link *or* link directly.

**How it works in Tracker today:** the choice is stored on the offer ( `linking_type` ) and shown again when you edit, but the tracking server never reads it. Every click must go through `/click` , which always redirects. There is no landing-page JavaScript click SDK yet, so direct-linked traffic would **not** be tracked.

#### REDIRECT + DIRECT

No behaviour change. Partners still need the tracking link for clicks to count.

#### REDIRECT ONLY

Same — this is how Tracker always works today.

### Support Deep Links

SAVED ONLY

**Purpose:** lets a partner send the user to a *specific* page on the advertiser's site (a product page, a category, a sale page) instead of the offer's single default landing page — without the advertiser having to create a separate Offer URL for every page. Typically the partner appends the target address to the tracking link, e.g. `/click?offer_id=12&pub_id=7&url=https://shop.example.com/product/123` , and the tracker redirects there (usually only if the domain matches the offer's domain, to stop it being abused as an open redirect).

**How it works in Tracker today:** the checkbox is saved ( `deep_link_enabled` ) but `/click` ignores any `url` parameter and always redirects to the offer's **Default Landing Page URL** (with macros filled in). Partners who need different pages should use separate Offer URLs/offers for now.

#### CHECKED

Stored only. Deep-link parameters on the tracking link are ignored.

#### UNCHECKED

Default landing page always — the same as today.

### Tracking Domain

ENFORCED

The domain partners' links are built on. The tracking server identifies the network (tenant) from this domain (the `Host` header), so it must be **active and verified**. For local testing the domain is `localhost` , and links use `http://localhost:4002` .

## 3. Postback Configuration (Image 2)

This tab gives the **advertiser** the URL to call when a conversion happens:

```
http://localhost:4002/postback?click_id={click_id}&txn_id={txn_id}&status=approved&secure_code=YOUR_CODE
```

| Parameter           | Meaning                                                                                                                                                                                                                                                                              |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| click_id (required) | The click id the landing page received in the redirect. Also accepted as <code>cid</code> / <code>clickid</code> .                                                                                                                                                                   |
| txn_id              | The advertiser's own order/transaction id. Used to block the same order being counted twice. Also <code>transaction_id</code> / <code>tid</code> .                                                                                                                                   |
| status              | <code>approved</code> / <code>confirmed</code> / <code>sale</code> / <code>success</code> / <code>1</code> → approved; <code>rejected</code> / <code>declined</code> / <code>reversed</code> / <code>cancelled</code> → rejected; <b>anything else or missing</b> → <b>pending</b> . |
| event               | Conversion event/goal name (e.g. <code>purchase</code> ). If missing, the offer's Base Conversion Event Name is used.                                                                                                                                                                |
| payout / revenue    | Optional overrides of the partner payout / advertiser revenue for this one conversion.                                                                                                                                                                                               |
| sale_amount         | Sale value — used when the offer's revenue type is Percentage or Mixed.                                                                                                                                                                                                              |
| secure_code         | The security code (below). Also <code>security_code</code> .                                                                                                                                                                                                                         |

The tracker answers with JSON — useful when testing: `{"status":"approved","conversion_id":"..."}`, or `"pending"`, `"rejected"`, `"duplicate"`; HTTP **404** `click_not_found`; HTTP **403** `invalid` or `missing secure_code`.

## Per-Offer Security Code PARTIAL

**What it is:** a secret token the advertiser must include as `secure_code=...` in every postback. It proves the postback really comes from the advertiser and not from someone who simply knows (or guessed) a `click_id` — without it, anyone holding a `click_id` could create conversions and earn payouts.

### How it works:

- There is a **network-wide** code (Settings > Security) and an optional **per-offer** code. If the offer has its own code it **overrides** the network code; if empty ("using network default") the network code applies.
- Generate** creates a random code; **Regenerate** replaces it — the advertiser's old URL immediately stops working, so share the new URL first; **Remove** falls back to the network code. Changes apply to the tracker instantly.
- The code is compared securely (timing-safe). Wrong or missing code → the postback is refused with HTTP 403 and no conversion is created.

**Important limits (current code):** (1) The check applies only to the S2S `/postback` endpoint — `/pixel` and `/iframe` conversions are **never** checked. (2) If **neither** a per-offer nor a network code is set, **no check happens at all**. Always set at least a network code, and prefer S2S postbacks for offers with real money.

## Publisher Postback — "Fire Partner Postback" ENFORCED

**What a publisher (partner) postback is:** the reverse direction of the advertiser postback. When a conversion is approved, Tracker calls the **partner's** server so the partner's own tracking/ad platform learns about the conversion (to optimise campaigns, pay their sub-affiliates, etc.).

This checkbox is the **same setting** as "Fire Partner Postback" on the Revenue & Payout tab (explained in §4) — changing one changes the other.

**Where the URL comes from:** each partner's postbacks are configured on the partner (Partners > *partner* > Postbacks), not on the offer. A postback can apply to one partner + one offer, a partner for all offers, or an offer for all partners, optionally for one event only. The most specific match is used. Supported macros:

`{click_id}` `{conversion_id}` `{offer_id}` `{publisher_id}` `{event}` `{payout}` `{currency}` `{txn_id}` `{sub1}` ... `{sub5}`

**Delivery:** a background worker sends it (GET, or POST with JSON), 10-second timeout, up to 6 attempts with increasing wait; every attempt is logged and visible in **Reporting > Partner Postback**. Internal/private addresses are blocked for safety.

**Not supported yet:** postback *delay*, HTML/pixel-type partner postbacks, CPC-level postbacks, and a `{revenue}` macro. If a partner has no postback configured, the log shows `no_postback_configured` — that is not an error.

## 4. Revenue & Payout (Events) (Image 3)

### Base Conversion Event Name ENFORCED

The event name recorded when the advertiser's postback does not send `event=` (e.g. `install`). It also decides which partner postback (event-specific ones) matches.

### Fire Partner Postback ENFORCED

**Why:** some partners want to be notified of every approved conversion; for others (or for test offers) you want silence.

#### CHECKED (DEFAULT)

When a conversion is approved — automatically, or later by manual approval — the partner postback job is queued and sent as described in §3. Not sent for pending/rejected conversions or throttled ones.

#### UNCHECKED

No partner postback is ever sent for this offer. The conversion, the ledger entries and the partner's earnings are still recorded normally — only the notification is skipped.

## Manually Approve Conversions **ENFORCED**

**Why:** for offers where the advertiser needs to validate leads/sales first (fraud review, chargebacks, quality checks) before money moves.

### CHECKED

Any conversion that would have been

#### APPROVED

is stored as

#### PENDING

with reason `manual_approval_required`.

#### NO LEDGER ENTRY

(no partner earning, no advertiser billing) and

#### NO PARTNER POSTBACK

. An admin/finance user then opens

#### REPORTING > CONVERSION

and clicks

#### APPROVE

→ status becomes approved, the ledger earning + billing are written, and the partner postback fires (if enabled).

#### REJECT

→ status rejected; nothing is paid.

### UNCHECKED (DEFAULT)

The `status` the advertiser sends decides: `status=approved` → approved immediately with ledger entries and partner postback; missing/unknown status → pending (and it still needs manual approval later).

Notes: only **pending** conversions can be approved. Rejecting an already-approved conversion writes *reversal* ledger entries (money is never edited or deleted). The manual Approve action does not re-apply the throttle or customer-value rules that the automatic path applies.

## Allow Duplicate Conversions **ENFORCED**

**Why:** some offers legitimately convert many times per click (e.g. an app with several in-app purchases, a shop where the same visitor orders twice). Others must count only one conversion per click (e.g. one install, one lead).

### CHECKED (DEFAULT)

The same click can produce several conversions

#### AS LONG AS EACH HAS A DIFFERENT `TXN_ID`

(or a different event). The same `txn_id` is

#### ALWAYS

treated as a duplicate, whatever this setting says.

### UNCHECKED

Only the

#### FIRST

conversion per click counts. Any later postback for that click — even with a new `txn_id` — gets `"status": "duplicate"` and nothing is recorded.

Extra protection that is always on: more than 10 different transaction ids on one click within 10 minutes are rejected as abuse.

## Base Revenue — Revenue Action / Type **PARTIAL**

**Revenue Type** is enforced: **Fixed** = the RPA amount per conversion; **Percentage** = X% of the `sale_amount` the advertiser sends; **Mixed** = fixed + percentage. **Revenue Action** "Impression" or "Click" and **Price Per Product** are saved only — revenue is booked on conversions only.

## 5. Attribution (Image 4)

These settings decide which conversions count, how fraud is scored and how clicks are returned. The screenshot shows the bottom half of the tab; the settings above it are included too.

### 24metrics Tracker **SAVED ONLY**

Stores your 24metrics tracker ID on the offer for reference. Tracker does not call 24metrics. Tracker's own fraud checks (datacenter IP, click velocity, conversion spikes) run on every click regardless.

### Enable IPQualityScore Fraud Detection **PARTIAL**

**Why:** to flag proxy/VPN/Tor/bot traffic per click.

#### YES

If IPQualityScore is connected in Integrations, each click's IP is scored in the background (never slows the redirect); the fraud score and flags (proxy, vpn, tor, bot) are saved on the click and visible in the Click report. It **DOES NOT BLOCK** clicks or reject conversions automatically.

#### NO

No external lookup; only the built-in fraud signals.

### Apply Throttle Rate **ENFORCED**

**Why:** also called "shaving" or a hold-back. A network uses it to keep a share of conversions (e.g. to compensate for low-quality traffic or as agreed margin). Use with care and in line with your partner agreements.

#### YES (E.G. 10%)

Each approved conversion has a 10% random chance of being throttled: it stays **APPROVED** and the **ADVERTISER IS STILL BILLED** (revenue recorded), but the **PARTNER PAYOUT BECOMES 0**, reason throttled, and **NO PARTNER POSTBACK** is sent. Must be between 0 and 100%.

#### NO

Every approved conversion pays the partner.

### Enable Click to Conversion Time **ENFORCED**

**Why:** conversions that happen impossibly fast after the click (bots, cookie stuffing, injected installs) or too long after it are suspicious.

#### YES

Two limits:

##### **MINIMUM LOOKBACK WINDOW (SECONDS)**

(default 5) and

##### **MAX. APPROVED CLICK TO CONVERSION TIME**

(1 h ... 30 days, or No maximum; default 6 h). A conversion that arrives sooner than the minimum is

##### **REJECTED**

( `click_to_conversion_too_fast` ); later than the maximum →

##### **REJECTED**

( `click_to_conversion_too_slow` ). Max must be greater than min.

#### NO

No timing check. The offer's normal attribution window still applies (conversions after it are rejected as `outside_attribution_window` ).

### Enable Server-Side Click **ENFORCED**

**Why:** some partners (apps, server-to-server integrations, push platforms) generate the click from their own server and need the `click_id` back instead of a browser redirect.

#### YES

Calling the tracking link with `&format=json` returns HTTP 200 with `{ "click_id": "...", "redirect_url": "..." }` instead of a redirect. The click is still recorded exactly like a normal click. Without `format=json` the link still redirects normally.

#### NO

`format=json` is ignored — always a 302 redirect.

### Enable Email Ownership **SAVED ONLY**

**Concept:** in email marketing, "ownership" attributes a conversion to the partner who owns the email subscriber (matched by email/hashed email) rather than by click. **Today:** stored on the offer; conversions are attributed by click only.

### Enable View-Through **SAVED ONLY**

**Concept:** credit a conversion to a partner whose ad was *seen* (an impression) even if the user never clicked, within a view-through window. **Today:** Tracker has no impression tracking, so this is stored only — conversions always attribute to clicks.

## 6. Creatives (Image 5)

**What a creative is:** the ready-made marketing material an advertiser provides for an offer so partners promote it correctly and on-brand — banners/images, HTML ad units, text links, email creatives (with subject/from lines), and videos. Tracker supports the types **image**, **html**, **link**, **email**, **video**.

### How it works:

- Click **Creative** on the offer's Creatives tab to add one (name, type, file/URL or HTML, optional email fields). Creatives can also be managed network-wide under Offers > Creatives and assigned to several offers.
- Each creative has a "**visible to partners**" flag and a status (active / deleted).
- Partners normally download the creative and use it with their tracking link (the creative's link should point to the tracking link, not the advertiser's site).

**Current limitation:** creatives are managed in the admin only. The **partner portal does not show creatives yet**, so partners must receive them from their account manager. Creatives do not affect tracking or payouts.

## 7. Email (Image 6)

These settings matter for **email-marketing offers** (partners who send emails to their own lists). Compliance rules (e.g. CAN-SPAM) require senders to remove people who unsubscribed from the advertiser — that is what suppression lists are for. All five are currently stored on the offer (and validated) but are **not shown in the partner portal** and do not affect tracking.

| Setting                                        | What it is for                                                                                                                                       | When Yes / No                                                                                                                                                     |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Enable Suppression File</b><br>SAVED ONLY   | A link (http/https) to the advertiser's suppression list — email addresses partners must remove ("scrub") from their list before mailing this offer. | Yes: URL field appears and is saved (must start with http/https). No: no list.                                                                                    |
| <b>Ezepo Enabled</b><br>SAVED ONLY             | Ezepo is a third-party suppression-list management service. Intended to sync the offer's suppression list with Ezepo.                                | No Ezepo integration exists — nothing is synced either way.                                                                                                       |
| <b>Optizmo Suppression List</b><br>SAVED ONLY  | Optizmo is another suppression-list platform. Here you store the Optizmo list / access key id for the offer.                                         | Yes: list-id field appears and the page shows whether Optizmo is connected in Integrations. The integration currently only stores the key; no list is downloaded. |
| <b>Enable Email Instructions</b><br>SAVED ONLY | Free-text mailing rules for partners: approved From lines, approved Subject lines, sending rules, forbidden words.                                   | Yes: text box appears and is saved. No: no instructions.                                                                                                          |
| <b>Enable Email Opt-out</b><br>SAVED ONLY      | The advertiser's unsubscribe/opt-out link that partners must include in every email.                                                                 | Yes: URL field (http/https) is saved. No: none.                                                                                                                   |

## 8. Step-by-step testing guide

This walks through one complete flow on the local setup, then (§9) tests each setting. Use a **test advertiser, test partner and test offer** — never real ones — because every approved conversion writes permanent ledger entries.

### Before you start

- Docker: Postgres, Redis running ( `docker compose up -d` ). Backend: `cd api-backend; npm.cmd run dev` (all 5 services incl. **workers** — without workers clicks never reach the reports and postbacks are never sent). Frontend: `cd frontend; npm.cmd run dev` , open `http://localhost:5173` and log in as admin.
- In PowerShell use `curl.exe` (not `curl` , which is an alias for something else). Put URLs in double quotes because of `&` .
- A tracking domain `localhost` must be active + verified (it already exists in the current database). Tracking links locally look like `http://localhost:4002/click?...` .
- Have a free request-catcher ready to see partner postbacks, e.g. a unique URL from **webhook.site**.

### Step 1 — Create the Advertiser

1. Advertisers > Create. Name "QA Advertiser", status **Active**, currency USD. Save.

**Verify:** it appears in the Advertisers list with a numeric ID (ref).

### Step 2 — Create the Partner (Publisher)

1. Partners > Create. Name "QA Partner", status **Active**. Save and note its numeric ID.

**Verify:** shown in the Partners list. (Note: the tracker does not currently block suspended partners, so don't rely on status to stop traffic.)

### Step 3 — Create the Offer

1. Offers > Create. Advertiser = QA Advertiser, status **Active**.
2. **Default Landing Page URL** — include macros so you can see the click id arrive:  
`https://example.com/landing?click_id={click_id}&sub1={sub1}`
3. Payout (partner) e.g. **5**, Revenue (advertiser) e.g. **8**, Revenue type Fixed.
4. Tracking Domain = `localhost` . Visibility: Public (simplest) — or Private/Require approval and do Step 4.
5. Leave Manual Approval off, Allow Duplicates on, Fire Partner Postback on, no caps, no targeting (add those later in §9).

**Verify:** Offer detail shows the numeric offer ID and the Postback Configuration tab shows the advertiser postback URL.

### Step 4 — Give the Partner access (private / approval offers only)

On the offer, approve QA Partner (or approve their application). Without approval, clicks on a non-public offer are diverted to the fallback URL.

### Step 5 — Set a security code

Postback Configuration > Per-Offer Security Code > **Generate**. Copy the full advertiser postback URL — it now contains `&secure_code=...` .

## Step 6 — Configure the Partner postback

1. Partners > QA Partner > Postbacks > Add. URL:

```
https://webhook.site/<your-id>?click={click_id}&conv={conversion_id}&payout={payout}&txn={txn_id}&s1={sub1}
```

2. Scope: this partner (optionally this offer), status Active, method GET.
3. Use the **Postback Test** button — webhook.site should receive a request with sample values.

## Step 7 — Build the tracking link

```
http://localhost:4002/click?offer_id=<OFFER_REF>&pub_id=<PARTNER_REF>&sub1=qa-test-1
```

Numeric IDs or full UUIDs both work. `aff_id` / `p` also work for the partner, `o` for the offer; `sub1` ... `sub5` and `source_id` are passed through.

## Step 8 — Make a click

```
curl.exe -i "http://localhost:4002/click?offer_id=12&pub_id=7&sub1=qa-test-1"
```

**Expected:** HTTP/1.1 302 and a Location: header like `https://example.com/landing?click_id=9f3c...&sub1=qa-test-1`. **Copy the click\_id.** (Opening the link in a browser also works — read the click\_id from the address bar.)

**Verify:** Reporting > Click (Run Report) shows the click within a few seconds with offer, partner, device/OS/browser, Unique = Y. A second click from the same machine shows Unique = N.

If you get 429 "Too many requests", wait one minute — the tracking rate limiter currently counts much more strictly than configured (known issue).

## Step 9 — Fire the conversion (advertiser postback)

```
curl.exe "http://localhost:4002/postback?click_id=<CLICK_ID>&txn_id=ORDER-1001&status=approved&secure_code=<CODE>"
```

**Expected response:** `{"status": "approved", "conversion_id": "..."}`

**Verify:**

- Reporting > Conversion: a row with status **Approved**, payout 5, revenue 8, txn ORDER-1001, and the click date.
- Reporting > Partner Postback: a delivery row with HTTP 200, and webhook.site received `click=...&payout=5.000&txn=ORDER-1001&s1=qa-test-1`.
- Dashboard / Offer report: clicks 1, conversions 1, payout 5, revenue 8, profit 3.

## Step 10 — Approval flow

1. Edit the offer: tick **Manually Approve Conversions**. Save.
2. Make a new click (Step 8) and fire a postback with a new txn ( `ORDER-1002` ).
3. **Expected:** response `"status": "pending"` ; Conversion report shows **Pending** (reason `manual_approval_required`); **no** new partner postback; partner balance unchanged.
4. In Reporting > Conversion click **Approve** on that row → status Approved, partner postback is sent, balance increases by 5.
5. Repeat with another conversion and click **Reject** → status Rejected, no money, no postback.

## Step 11 — Partner balance and payout

1. Check the partner's balance: Partner portal › Earnings, or the admin API `GET /api/finance/publishers/<partner-uuid>/balance`. Expected:  $5 \times$  number of approved, non-throttled conversions.
2. Create a payout run. There is **no button for this in the UI yet**; use the API (as an admin/finance user). Get your access token from the browser: DevTools › Application › Local Storage › `tracker.session.v2` › `access_token`.

```
curl.exe -X POST "http://localhost:5173/api/finance/payouts/batches" -H "Authorization: Bearer <TOKEN>" -H "Content-Type: application/json" -d "{\"publisherIds\":[\"<partner-uuid>\"],\"note\":\"QA payout\"}"
```

**Verify:** `GET /api/finance/payouts/batches` lists the batch; the partner balance returns to 0; `GET /api/finance/ledger` shows the earning (credit) entries and one payout (debit) entry. Ledger rows are never edited or deleted.

Run payouts only for test partners and only once at a time — two runs started at the same moment can both pay the same balance (known issue).

## Step 12 — Advertiser side

Reporting › Advertiser shows QA Advertiser's conversions and revenue (8 per approved conversion, including throttled ones). Advertiser invoices (Advertisers › Invoices) can be created from that revenue. The advertiser portal needs an advertiser login, which doesn't exist in the current local database.

## 9. Settings test matrix

For each test: change the setting, Save, make a **new click** (Step 8) and fire a postback (Step 9) unless stated. Use a new `txn_id` each time.

| Setting / case               | How to test                                                                              | Expected result                                                                                                                                   |
|------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Status missing               | Postback without <code>&amp;status=</code>                                               | "pending" ; no ledger, no partner postback.                                                                                                       |
| Wrong security code          | <code>secure_code=WRONG</code>                                                           | HTTP 403 <i>invalid or missing secure_code</i> ; nothing recorded.                                                                                |
| Unknown click                | <code>click_id=doesnotexist</code>                                                       | HTTP 404 <i>click_not_found</i> .                                                                                                                 |
| Same txn twice               | Send the same postback twice                                                             | 2nd response "duplicate" ; only one conversion.                                                                                                   |
| Allow Duplicates ON          | Same click, txn A then txn B                                                             | Two approved conversions.                                                                                                                         |
| Allow Duplicates OFF         | Same click, txn A then txn B                                                             | A approved; B → "duplicate" .                                                                                                                     |
| Manual Approval ON           | Normal approved postback                                                                 | "pending" until Approve in Reporting > Conversion (see Step 10).                                                                                  |
| Fire Partner Postback OFF    | Approved postback                                                                        | Conversion approved, balance increases, <b>no</b> new row in Partner Postback report / webhook.site.                                              |
| Throttle 100%                | Approved postback                                                                        | Approved, revenue 8, payout 0, reason throttled, no partner postback. (Set back to off afterwards.)                                               |
| Click-to-Conversion min 60 s | Postback immediately after the click                                                     | "rejected" , reason <code>click_to_conversion_too_fast</code> . Wait 60 s with a new click → approved.                                            |
| Server-Side Click ON         | <code>curl.exe ".../click?offer_id=...&amp;pub_id=...&amp;format=json"</code>            | HTTP 200 JSON <code>{click_id, redirect_url}</code> . With the setting OFF → 302.                                                                 |
| Percentage revenue 10%       | Postback with <code>&amp;sale_amount=200</code>                                          | Revenue 20.                                                                                                                                       |
| Payout override              | Postback with <code>&amp;payout=7</code>                                                 | Conversion payout 7.                                                                                                                              |
| Offer paused                 | Set offer status Paused, then click                                                      | Redirect to fallback URL (or 204), no click credited.                                                                                             |
| Country targeting            | Include Country = India, then click                                                      | Locally the GeoIP database is missing, so geo rules are <b>skipped</b> (click allowed). Test on a server with GeoIP: non-Indian IPs are diverted. |
| Browser targeting            | Include Browser = Firefox, click from Chrome                                             | Click diverted to the Fail Traffic URL; from Firefox → normal redirect.                                                                           |
| Daily click cap              | Caps > daily click cap 2, then 3 clicks                                                  | 3rd click diverted.                                                                                                                               |
| Pixel conversion             | <code>http://localhost:4002/pixel?click_id=...&amp;txn_id=...&amp;status=approved</code> | 1×1 image; conversion recorded (note: no security code check on pixel).                                                                           |

## 10. Final verification checklist & known gotchas

---

- ☐ Advertiser, Partner and Offer exist and are Active; offer uses tracking domain `localhost` .
- ☐ Click returns 302 with the `click_id` in the landing URL; click appears in Reporting > Click.
- ☐ Approved postback returns `"approved"` ; conversion appears with correct payout/revenue.
- ☐ Partner Postback report shows a successful delivery; the catcher URL received the macros.
- ☐ Manual approval holds conversions as pending; Approve/Reject behave as described.
- ☐ Duplicate, security-code, click-to-conversion and throttle cases match §9.
- ☐ Partner balance = approved non-throttled payouts; payout run moves it to 0 and adds a ledger debit.
- ☐ Offer / Partner / Advertiser reports agree on clicks, conversions, payout, revenue, profit.

### Known gotchas while testing

- **Workers must be running** — clicks appear in reports and partner postbacks are sent only by the worker service.
- **Country/Region/City show "—" locally** because the GeoIP database isn't installed; geo targeting is skipped, not enforced.
- **Rate limit 429** on repeated clicks/postbacks from one machine — wait a minute.
- **Report date range is in UTC**; timestamps are displayed in the chosen timezone.
- **Pixel/iframe conversions skip the security code**, and status/payout can be passed in the URL — don't give pixel URLs to untrusted parties.
- **Suspended partners/networks are not blocked** by the tracker yet.
- "Saved only" settings in this guide will not change any test result.

Source references: `tracking/app.ts`, `tracking/conversions/record.ts`, `workers/processors/outbound-postback.ts`, `lib/ledger/ledger.ts`, `dashboard/finance/*`, `frontend offerForm/SettingsPanels.tsx`, `OfferEdit.tsx`, `lib/trackingLinks.ts`.